

Highlights

Blockchain access control for agricultural edge IoT: a 61-day Hyperledger Fabric field study and implementation postmortem

Forcha Glen Beloa, Michael Ekonde Sone, Elie Fute Tagne, André Paul Mvogo Asse

- A 61-day field deployment recorded 209,000 authorization decisions.
- Eight counterbalanced runs found no detected peer-campaign order effect.
- Logical peer multiplicity was associated with 388 ms lower write latency.
- A corrected chaincode package removes two audited policy defects.
- Missing episode keys prevented end-to-end revocation-exposure analysis.

Blockchain access control for agricultural edge IoT: a 61-day Hyperledger Fabric field study and implementation postmortem^{*}

Forcha Glen Beloa^{a,*}, Michael Ekonde Sone^a, Elie Fute Tagne^a and André Paul Mvogo Asse^b

^aDepartment of Computer Engineering, Faculty of Engineering and Technology, University of Buea, Buea, Cameroon

^bÉcole Nationale Supérieure Polytechnique de Douala, Université de Douala, Douala, Cameroon

ARTICLE INFO

Keywords:

Hierarchical role-based access control
Hyperledger Fabric
Agricultural IoT
Edge computing
Performance measurement
Certificate revocation
Reproducible research

ABSTRACT

Blockchain access control for Internet of Things systems is usually evaluated using short laboratory measurements of aggregate throughput and latency, leaving where delay falls and its long-term operational validity uncertain. We report a 61-day Hyperledger Fabric 2.5 deployment of a hierarchical role-based access control (HRBAC) framework on a 2 ha farm in Sfax, Tunisia (50 sensors, 4 gateways, 4 zones, 209 000 authorization decisions), a follow-up peer-scaling campaign, and an implementation postmortem. A chaincode timing field recorded an authorization-associated span averaging 320 ms of the 1 205 ms write path. Across 8 counterbalanced runs, increasing configured logical peer multiplicity from 4 to 32 on four fixed hosts was associated with a 388 ms drop in the uninstrumented remainder; the recorded span differed by only 4.78 ms between extremes. A separate, interleaved concurrency benchmark found that the complete access-control-enabled path averaged 9.6% lower throughput than the comparison path, with no condition-by-concurrency interaction detected across a tenfold range.

A post-deployment audit found three defects these measurements could not expose, each requiring cross-component testing: the deployed role hierarchy left the intended auditor separation unenforced; 131 795 readings reconstructed from two of three residues were mathematically incorrect; and gateway signature verification checked a different message than the sensor signed, accepting unsigned readings by default. All three are corrected: two in the released code, the role-hierarchy fix in a separately tagged package that leaves the measured code unmodified. The performance results characterize the deployed implementation, not the corrected security design. We release the traces, execution schedules, both chaincode artifacts, and the pipeline that regenerates every number here.

1. Introduction

A precision farm is a multi-stakeholder system before it is a technical one. Soil probes, weather stations and irrigation actuators produce readings continuously [52], and the people who consume those readings want different and partly conflicting things from them: farmers need operational control, agronomists need history but no actuation rights, certification bodies need compliance evidence without visibility into commercially sensitive agronomic detail, supply-chain partners need provenance but not real-time operations, and regulators need environmental records.

Handing this authorization matrix to a single server concentrates the risk in one place. An attacker who takes that server gains not only the data but, on a farm with networked valves, the irrigation itself [6], and the audit log that would reconstruct the incident is written by the machine that was compromised.

^{*} The trace dataset, chaincode, firmware, formal specification and the analysis pipeline that regenerates every number, table and figure in this article are available at <https://github.com/orchag/blockchain-fet>.

*Corresponding author

thunbeils@gsafrail.net (F.G. Beloa);
ekonde.sone@gmail.com (M.E. Sone); eliefute@yahoo.fr (E.F. Tagne); mvgogoasseandre@gmail.com (A.P.M. Asse)
ORCID(s): 0009-0003-1972-6100 (F.G. Beloa)

Surveying agricultural IoT deployments, Ferrag et al. [10] found most lacked fine-grained access control at all, and Kamilaris et al. [18] and recent convergence surveys [33] find blockchain work in agriculture concentrated almost entirely on post-harvest supply-chain tracking rather than pre-harvest authorization. Recording role assignments, revocations and access decisions on a permissioned ledger can reduce dependence on a single policy database and make authorized changes auditable across participating organizations; it does not eliminate trust in the certificate authority, administrators, gateway software, endorsement configuration or chaincode implementation. What is unsettled is the cost of the part it does change, and specifically where that cost falls.

The constraints behind the design in Section 3 are not incidental to the measurement; they are why a field deployment answers questions a laboratory benchmark cannot. Constrained sensor energy budgets push caching and enrollment logic to the edge gateway; intermittent rural connectivity, not injected fault, is why the revocation and outage behaviour of Sections 6.5 and 6.8 includes a certificate revoked while its gateway is disconnected, a failure mode a short lab trial would not encounter; and LoRa's low bit rate and eFuse-only

key storage set the honest bounds behind the residue-coded transmission and signature defects of Sections 7.2 and 7.3. None of this is unique to agriculture, but the combination, long-running, energy-constrained, intermittently connected, physically exposed, is what makes a multi-week field deployment a different instrument than a data-centre benchmark of the same chaincode.

Among the systems surveyed in Table S1 (supplementary material), evaluation is typically a laboratory or simulated run of hours to days, reporting peak throughput and mean latency that aggregate over a pipeline of at least four separable stages, so an engineer told a transaction takes 1 220 ms cannot tell whether to rewrite the policy engine, restructure endorsement, replace the ordering service, or accept the figure as a consensus floor. Existing evaluations rarely combine long-duration field observations with an implementation-level authorization timing span, controlled configuration experiments and a post-deployment audit; that combination, not any single measurement, is the gap this paper addresses. Attributing time to Fabric's internal stages needs one transaction timestamped at each hand-off, which our instrumentation does not do, so we report a coarser accounting instead: the chaincode logs a timing field on every write, and because its own start and stop boundaries were not documented at deployment time we treat it as an authorization-associated span rather than a stage measurement, with endorsing-peer count acting as a lever on what remains. Abang et al. [1], reviewing Fabric latency modelling for IoT, make a closely related point: the literature models latency in the abstract but rarely connects those models to a specific deployed use case measured long enough for its slow-moving parts to misbehave. We do not supply that stage-level measurement either; what we add is a long-duration field record against which the recorded span holds up, plus a specification audit the deployed benchmark measurements did not prompt, which turned up three defects that had run for the whole deployment undetected.

The intuitive expectation turns out to be wrong, which is the main reason we think the measurement is worth reporting. Access control is the component under study, so it is natural to assume it dominates the cost. The recorded span does not: it is a modest and remarkably stable fraction of the write path, and the larger, more variable share sits in the layer that access-control papers usually treat as background infrastructure. A second gap is temporal: certificate lifecycle management, revocation propagation and enrollment recovery are the mechanisms most likely to fail in the field and least likely to be exercised by short-duration benchmark measurements, and a revocation pipeline that behaves perfectly across fifty laboratory trials can still show a materially different delay distribution across two months of intermittent rural connectivity.

These questions structure the rest of the paper:

RQ1 What latency and throughput differences are observed between the tested access-control-enabled configuration and the access-control-free comparison configuration?

RQ2 How is configured logical peer count associated with client-observed write latency under the deployed topology?

RQ3 Which implementation and observability failures survived the field deployment, and why did the existing tests not detect them?

We built an HRBAC framework on Hyperledger Fabric 2.5 for the constraints of agricultural edge deployment, ran it on a working farm for 61 days, and instrumented the authorization-associated span and the end-to-end transaction path. Four bounded contributions follow, each naming which evidence it rests on – the field deployment, a later controlled laboratory campaign, or the code audit – and none extending its claim beyond that source: the field observations characterize the deployed system as run; they are not measurements of the corrected implementation or evidence of universal blockchain scalability. **A 61-day field characterisation**, 209 000 authorization decisions over 50 sensors, 4 gateways and 4 zones, addressing RQ1 (Section 6). **A controlled comparison across tested configurations**: the recorded authorization-associated span averages 320 ms, 26.6% of the 1 205 ms write path, and a replicated comparison of the 4- and 32-peer configurations finds it changes by only 4.78 ms while the remaining latency falls by 388 ms, addressing RQ2; neither the span's internal boundaries nor the remainder's internal stages are separately identified (Sections 6.1 and 6.4). **An implementation postmortem**: an audit of the implementation against its own specification found three defects that survived 61 days of operation, plus one observability gap, addressing RQ3, each a failure mode the deployed benchmark measurements and component-level tests did not detect and, together, the part of this work most likely to transfer to other deployments (Section 7). **A reproducible artifact and cross-layer testing recommendations**: the trace dataset, chaincode, firmware, formal specification and analysis pipeline are released under a tagged version (Section 5.5), together with the seam-level checks the postmortem motivates, a permission table generated from the deployed policy code rather than transcribed, an encoder test pinned to its recovery bound, and a verifier test pinned to the firmware's exact byte layout (Section 7.5).

Section 2 positions the work. Section 3 presents the access-control model and the enrollment protocol, Section 4 the implementation, Section 5 the measurement methodology, Section 6 the results, Section 8 what follows from them for system design, and Section 9 concludes.

2. Related work

2.1. Access control for constrained devices

Role-based access control as formalised by Sandhu et al. [36] and standardised by NIST [11] is still the reference model for multi-stakeholder authorization, and its IoT adaptations share a structural assumption that becomes awkward in the field: some trusted decision component must be reachable when access is requested. RBAC extensions for IoT [49], attribute-based evaluation [15, 38, 37] and capability-token delegation [14] place that trust in a policy engine, an attribute authority and a credential issuer respectively, retaining a central enforcement point [49, 38], distributing revocation into tokens rather than solving it [14], or splitting the model between humans and devices while leaving the role store centralised [3]. Sciancalepore et al. [37] quantify what open-standard authorization stacks cost on constrained hardware, the empirical case for moving policy evaluation off the sensor entirely; Novo [27] argued early that the role store itself belongs on a distributed ledger. We take that as given and ask what it costs.

2.2. Blockchain-based access control and how it is evaluated

Capability-token systems on public chains established feasibility without practicality: FairAccess [29] encodes grants as Bitcoin-like transfers with confirmation windows measured in minutes, and BlendCAC [48] uses ERC-721 tokens with per-transaction fees no sensor duty cycle can absorb. Neither addresses role hierarchy, geographic scoping or seasonal expiry, which agriculture actually needs.

Permissioned platforms remove the latency and fee obstacles, and Fabric-based IoT work has grown accordingly: testbed architectures and performance models [39, 30, 19], trust and reputation layers [31], contract-enforced domain policy on cloud-hosted nodes where memory and thermal limits do not apply [44], and a data marketplace at scale [13]. All report aggregate throughput and latency from a controlled setting. Two are closer to our method: Idrissi and Palmieri [17] give an end-to-end Fabric latency breakdown on constrained ARM hardware in a healthcare context, the nearest methodological precedent for the partition in Section 6.1.1, and Tzenetopoulos et al. [43] establish a Raspberry Pi 3B+ reference point discussed in Section 8.

Supplementary Table S1 summarises how these and further systems [51, 5] are evaluated. The pattern is consistent: laboratory testbeds or simulation, runs of hours or days, aggregate reporting. We have not found prior work that times an authorization-associated span separately from the rest of the write path on a field deployment of this duration, or that reports revocation behaviour over a multi-week window; that combination, rather than any single mechanism in Section 3 or a

validated stage-by-stage decomposition, is the contribution.

2.3. Agricultural IoT and edge infrastructure

Blockchain in agriculture has concentrated on traceability after harvest [42, 22, 5], where the unit of interest is a shipment rather than a zone-scoped actuation. Safeer et al. [34] combine computer vision, IoT and blockchain for climate-smart cultivation and, like most pre-harvest work, treat authorization as an implementation detail rather than a measured component. Liu et al. [23] and Wang et al. [45] reduce processing latency at the agricultural edge without addressing the authorization layer above it, and Mohanta et al. [24] apply smart-contract enforcement in smart grids, a neighbouring domain with comparable device constraints and a similarly centralised starting point.

On identity infrastructure, W3C Decentralized Identifiers [46] and gateway-assisted verification for constrained devices [9, 40] offer an eventual migration away from X.509, treated here as future work rather than a deployed alternative. On the radio side, Adelantado et al. [2] characterise the LoRaWAN capacity limits that bound achievable sensor density per gateway, a constraint visible in the energy results (Supplement S6).

3. Access-control design

3.1. Deployment structure and threat model

The deployment has three tiers, shown in Figure 1. ESP32-WROOM-32 sensor nodes (dual-core Xtensa LX6 at 240 MHz, 520 kB SRAM) sample soil and environmental data, sign each reading with an Ed25519 key held in one-time-programmable eFuse, and transmit over LoRa. Raspberry Pi 4B zone gateways (quad-core Cortex-A72, 4 GB LPDDR4, TPM 2.0) receive those payloads, verify signatures, endorse Fabric transactions and cache policy decisions locally. A Fabric network of four peers, one per zone, three Raft orderers and CouchDB 3.2 for world state maintains the ledger. Gateways interconnect over a BATMAN-adv mesh [25] that reroutes around individual gateway failures and carries Fabric gossip and certificate revocation list (CRL) traffic between zones.

Following the permissioned-blockchain model in which every participant is credentialed [4], orderers and the certificate authority are operated by vetted institutional entities and treated as non-malicious; Raft's $2f + 1$ crash-fault tolerance [28] means the three-node cluster survives one orderer failure but not a reachable orderer behaving maliciously, a boundary Byzantine-tolerant ordering [50, 7, 16] would widen at higher consensus cost. Sensors, client devices and every wireless channel are treated as potentially compromised;

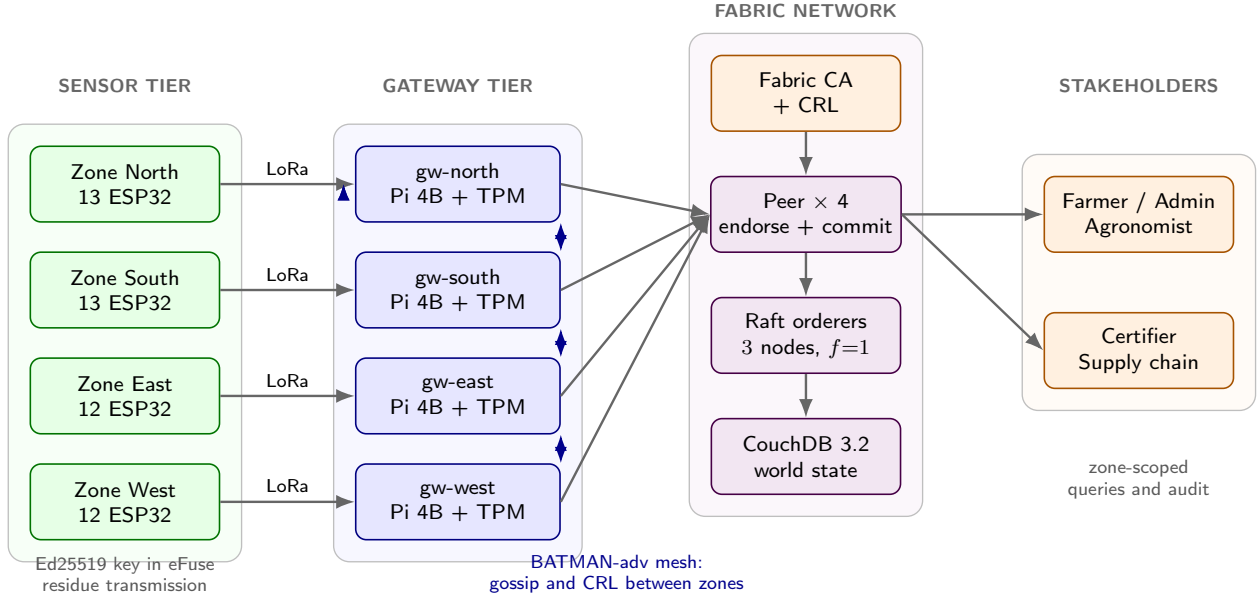


Figure 1: Deployment architecture. Sensors reach the ledger only through their zone gateway, which is also their registration authority during enrollment. Zone membership is carried in the device certificate and re-checked by chaincode on every transaction.

physical tampering and side-channel extraction against cryptographic accelerators sit outside the evaluated scope (Section 8).

The design and the evaluation below distinguish six properties: *authentication*, *authorization*, *integrity*, *freshness*, *auditability* and *availability*. The deployment gives usable evidence about authorization (Section 6.6), auditability (every decision writes an audit entry, Section 4.1) and availability (Section 6.8); it does not give usable evidence about authentication or integrity of sensor readings, because the signature path meant to bind a reading to its sensor did not function (Section 7.3), and freshness is architectural (nonce checking) rather than measured against an adaptive adversary.

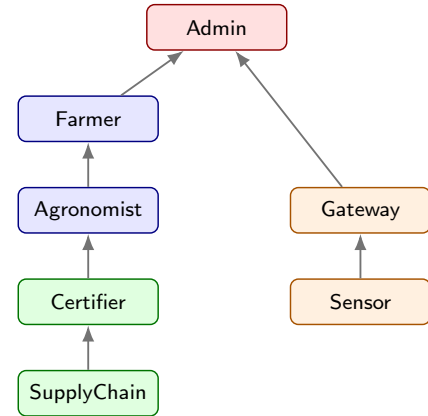
3.2. Role hierarchy

Following the treatment of hierarchical role administration in ARBAC97 [35], a partial order $\preceq$ over the role set R defines inheritance, with $r_i \preceq r_j$ iff $\text{perm}(r_i) \subseteq \text{perm}(r_j)$. The deployed hierarchy (Figure 2) is a human chain and a separate device chain, both below Admin:

$$\text{SupplyChain} \preceq \text{Certifier} \preceq \text{Agronomist} \preceq \text{Farmer} \preceq \text{Admin} \quad (1)$$

$$\text{Sensor} \preceq \text{Gateway} \preceq \text{Admin} \quad (2)$$

Separating auditors from operational staff is the requirement that motivated the model: an auditor should read compliance evidence without inheriting the agronomic permissions a farm treats as proprietary. Placing Certifier inside the human chain rather than



Arrows point from a role to one that inherits its permissions, as implemented in *roles.go*. Certifier sits inside the human chain, so Agronomist and Farmer also inherit *ReadAudit* (Section 7.1).

Figure 2: Seven-role hierarchy as deployed. Certifier lies on the human inheritance chain rather than beside it, so audit-log access is inherited upward by Agronomist and Farmer.

beside it defeats that: Agronomist and Farmer inherit Certifier's permissions, including *ReadAudit*, so the deployment did *not* confine audit-log access to auditors. We did not intend this and did not notice it until the implementation was audited against the manuscript for this article. We report the model as deployed, because that is the model the measurements describe; Table 1 gives the rows where intended, deployed and corrected policy disagree, and Section 7.1 the full account.

Table 1

Permission grants where the intended design, the deployed chaincode and the released correction disagree. Intended is independent requirements prose (Section 3.2), not derived from any code; deployed and corrected are both generated from `roles.go` (`chaincode/hrbac/`, tag `v09-deployed-historical`, and `chaincode/hrbac-corrected/`, tag `v09-corrected`). Every permission not listed here is identical across all three.

Permission	Role Intended	Deployed
ReadAudit	Agronomist	Yes
ReadAudit	Farmer No	Yes
ReadZone	Certifier No	Yes
ControlZone	Agronomist	Yes

The effective permission set of user u in a session is

$$\text{eff}(u) = \bigcup_{r \in \text{active}(u)} \bigcup_{r' \preceq r} pa(r'), \quad (3)$$

where $pa(r)$ are the permissions assigned directly to r and $\text{active}(u)$ the non-expired roles held by u . An access request (u, op, res, t) is granted iff

$$\begin{aligned} &\exists r \in \text{active}(u, t) : (op, res) \in \text{eff}(r) \\ &\wedge (\text{zone}(u) = \text{zone}(res) \vee \text{czt}(u, \text{zone}(res), t)), \end{aligned} \quad (4)$$

where $\text{czt}(u, z, t)$ holds iff u has a valid administrator-issued cross-zone token for z at time t , and $\text{active}(u, t) = \{r \mid \exists \text{assign}(u, r, t_s, t_e) : t_s \leq t \leq t_e\}$. The full effective-permission matrix this produces for all seven roles, generated from `roles.go` rather than transcribed, is Table S2 in the supplementary material; Table 1 below gives only the rows that differ between what was intended, what was deployed, and what the released correction changes.

3.3. Zone and temporal scoping

Geographic farm regions (North, South, East, West) define the zone model, enforced at three independent points: certificate SAN field, gateway enrollment (own zone only), and chaincode's ASN.1-decoded zone check on every transaction, chosen over PEM-text matching because the latter is spoofable by an attacker who controls the subject field. Cross-zone tokens are administrator-issued and recorded immutably, so a temporary grant expires on its own rather than needing a reissued certificate. Temporal constraints run in two independent layers, the chaincode's `ExpiresAt` check against the Fabric transaction timestamp and the Fabric MSP's `X.509 NotAfter` validation, so a seasonal worker loses access at expiry without administrator action, recorded as a new audit entry; GPS-sourced gateway time, NTP fallback, sits comfortably inside the ± 5 -minute enforcement window.

The decision procedure Equation (4) describes is given in full as Algorithm S4 in the supplementary

material. Two of its properties matter later: the work is bounded and independent of which operation was requested (hierarchy walked to depth at most three, zone check a single field comparison), the mechanism we would expect to produce similar costs across operations; and every path, including every denial, writes an audit entry, so `CheckAccess` is state-mutating and commits through ordering exactly as a sensor write does, with expired assignments revoked inside the same transaction that denies the request.

3.4. Cascading enrollment

No An ESP32 cannot hold the TLS handshake state direct interaction with a certificate authority requires (mbedtls on ESP-IDF needs roughly 70 kB of heap against a 520 kB SRAM budget [32]), so registration authority moves down the tier structure in three stages (Supplement, Figure S1 and Algorithm S1): administrator credentials, then gateway registration with the private key in TPM 2.0, then sensor enrollment with the gateway as local registration authority. The sensor's key is generated on-device and never transmitted, so compromising a gateway does not yield it, and re-provisioning a lost key needs physical access, since eFuse programming is irreversible – but the key is *not* read-protected: software signing copies it from eFuse into RAM each time, and setting the read-disable bit would break signing outright on this ESP32, which has no hardware Ed25519 engine. Read protection needs a different part (the ESP32-S3's HMAC and digital-signature peripherals); the eFuse here gives durability against reflashing, not extraction resistance against code running on the device, and Section 8 counts key extraction by an attacker with code execution as residual risk.

4. Implementation

4.1. Chaincode and transaction flow

The chaincode is written in Go 1.21 against `fabric-contract-api-go` and targets Fabric 2.5. Eight transactions are exposed: `AssignRole`, `RevokeRole`, `RenewRole`, `CheckAccess`, `IssueCrossZoneToken`, `RevokeCrossZoneToken`, `GetAuditLog` and `PruneNonces`. Supplement Figure S2 places the access-control check inside Fabric's `execute-order-validate` pipeline: an authorization denial is a successful chaincode response carrying `DENY` plus an audit write, so it is ordered and committed like any other transaction, and only an endorsement-level failure stops before the orderer.

Three implementation decisions shape the measurements. Every state-mutating call carries a UUID nonce recorded in world state, rejected on reuse before any state is read, which defeats replay inside the timestamp window. Every access attempt, granted or denied, writes an audit entry holding the caller's certificate hash, operation, resource, zone, decision and

reason, so the audit trail is a byproduct of enforcement rather than a parallel log that can drift from it. And `CheckAccess` revokes an expired assignment in the same transaction that denies the request, which makes even a read-intent check state-mutating and therefore endorsed rather than served as a query.

Peer configuration for the ARM64 gateways uses 50 transactions per block, a 500ms batch timeout, a 256MB state cache, endorsement restricted to zone-local peers, Snappy compression on LevelDB and 5-minute gRPC keepalive, targeting the write amplification that dominates latency on microSD-backed storage, within the parameter space Abang et al. [1] identify as governing Fabric latency.

4.2. Sensor firmware and residue-coded transmission

The sensor certificate lives in battery-backed RTC slow memory so it survives deep sleep, guarded by a CRC and a magic sentinel checked on each wake; a mismatch forces re-enrollment rather than transmission under an unverifiable identity. Signing uses wolfSSL's Ed25519 implementation: the firmware hashes the payload with hardware SHA-256 and signs the resulting 32-byte digest, a prehash construction that is neither standard Ed25519 (which hashes the message internally with SHA-512) nor Ed25519ph (which prehashes with SHA-512 under a distinct domain separator). Nothing in the deployment depends on the difference, but a reimplemention from the paper alone would not interoperate unless the prehash is reproduced. The main loop wakes every 30 minutes, verifies the cached certificate, samples the sensors, signs the payload, transmits and returns to deep sleep.

Readings are transmitted as residues rather than plaintext values: a value is reduced modulo the pairwise-coprime set $\{97, 101, 103\}$ and the three residues go out as separate single-byte packets, any two of which reconstruct the original by the Chinese Remainder Theorem [12] (Supplement S2). The admissible range is where this scheme goes wrong, and the deployed firmware got it wrong: recovery from an arbitrary *pair* is unique only below the smallest pairwise product of the moduli, not below their product (Section 7.2). The corrected firmware uses $\{253, 254, 255\}$, a pairwise bound of 64 262, and quantises soil moisture from twelve to eight bits so the packed value reaches 51 455 and stays inside it; keeping twelve bits would require all three residues and forfeit the redundancy entirely.

4.3. Gateway services and revocation propagation

Each gateway runs Fabric 2.5 from official ARM64 binaries. A Python enrollment service bound to the local interface accepts CSRs only from sensors in its own zone and rate-limits per source address, and a

policy cache with a 301-second nominal TTL absorbs repeated checks from the same sensor during a transmission burst. Revocation publication and cache invalidation (Supplement, Algorithm S3) is the mechanism whose behaviour we most wanted to observe over a long window, and one branch is the whole story of Section 6.5: for a reachable gateway the cache is invalidated by the CRL event itself, so staleness is bounded by gossip propagation and the nominal TTL never binds, while for an unreachable one the bound falls back to the scheduled refresh interval. In the intended design, gateway reachability determines whether event-driven invalidation is available or whether the scheduled refresh path is required; because the historical trace lacks complete episode identifiers and synchronised connectivity state, it cannot establish which branch was followed in any given observed case.

4.4. Formal specification

A three-role subset of the model is specified in TLA+ and checked with TLC [20, 26], finding no violation of the role-permission invariant under weak fairness. This is a sanity check on a small model, not verification of the deployed framework: it covers three of seven roles, omits zone and temporal predicates, and nothing connects it to the Go implementation, the binding gap given that the implemented hierarchy differs from the specified one (Section 7.1). The specification is in the artifact.

5. Measurement methodology

5.1. Site and workload

The system ran continuously for 61 days on a 2 ha experimental farm near Sfax, Tunisia, from 1 August to 30 September 2025 (coordinates throughout this article, 34.74°N, 10.76°E, locate the city at settlement granularity, not the farm parcel, per the Ethics section). Table 2 gives the configuration; Sfax is unforgiving in a way that is useful for this measurement, with ambient temperatures reaching 39.5°C and rural connectivity intermittent enough that the outage behaviour of Section 6.8 reflects ordinary operation rather than injected faults.

Each sensor reported on a 30-minute duty cycle, giving 2 400 sensor writes per day and 146 400 over the deployment; human and administrative traffic (farmer and agronomist queries, certifier audits, supply-chain reads, role operations) brings the total to 209 000 recorded authorization decisions, of which 4 860 (2.33%) were ordinary-operation denials.

5.2. Peer-scaling topology

Section 6.4 varies *configured logical peer count*, not physical capacity: increasing configured peers does not automatically reduce Fabric latency, since the effect depends on the endorsement policy, endorser selection, connection pooling, proposal fan-out, and peer CPU

Table 2
Deployment configuration and observed totals.

Parameter	Value
Site	2 ha farm near Sfax, Tunisia (city coordinates: 34.74°N, 10.76°E)
Duration	61 days, 1 Aug to 30 Sep 2025
Zones	4 (North, South, East, West)
Sensor nodes	50 ESP32-WROOM-32
Gateways	4 Raspberry Pi 4B (4 GB, TPM 2.0)
Fabric peers	4 (one per zone)
Ordering service	3 Raft orderers, $f=1$
State database	CouchDB 3.2
Sensor duty cycle	30 min
Peak ambient temperature	39.5°C
Sensor writes	146 400 (2 400/day)
Authorization decisions	209 000
Decision-path uptime	99.36%
Connectivity outages	3 (0.2–8.7 h, 9.3 h total)

and state-database load. Only the 4-peer configuration is the physically deployed network of Table 2, one peer per zone on its own Raspberry Pi; the 8-, 16- and 32-peer configurations run additional logical peers on the same four physical hosts, oversubscribed rather than added, so the comparison isolates endorser multiplicity under a fixed endorsement policy, not physical capacity. The full topology and routing table (endorsement policy, orderer topology, batch parameters, CPU/memory allocation, all identical across configurations) is Table S3 in the supplementary material.

A run is one complete workload execution at a fixed peer count, separated from the next by a 5-hour block-and-cooldown interval that restarts the Fabric peer and orderer processes and lets ledger and CouchDB state settle first, so between-run variation is execution-to-execution, not carried-over state; this is what `scripts/generate_peer_scaling_campaign.py` models (each run's mean drawn independently, samples within a run around that run's own mean), making the runs independent replicates rather than one implicitly correlated sequence.

5.3. Instrumentation

Nine trace streams were recorded: per-decision records (caller identity, role, operation, resource, zone, decision, denial reason, latency), per-transaction sensor records (signature validity, residues received, transmission energy), and dedicated campaigns for peer-scaling latency, throughput, LoRa energy in single-channel and residue modes, cryptographic timings, revocation lifecycle, authorization-boundary attempts and outages. Latency is measured at the gateway between submission and commit notification (endorsement through commitment, excluding only the LoRa hop); throughput is committed transactions

per measurement window; energy uses an INA219 sensor at 2 kHz. The baseline for overhead attribution (Section 6.3) is the identical network, workload and hardware with the access-control chaincode removed, which also removes the audit write and nonce check, so the difference estimates the cost of the complete access-control-enabled path, not an isolated permission decision.

5.4. Statistical treatment

Getting the independent experimental unit wrong is the most common way a measurement study overstates precision: transactions generated in the same run, day or gateway are correlated, and treating each as an independent replicate produces standard errors far smaller than the design supports. Table S4 in the supplementary material states, for every comparison in this article, what that unit is; the two campaigns discussed below use the independent run (8 per peer-count configuration, 5 per throughput cell) rather than the transaction.

Group comparisons use Welch's t -test, reported with the mean difference, its 95% confidence interval and Cohen's d ; comparisons across more than two independent groups use one-way ANOVA. Where an order effect is a live concern, a multiple regression on the relevant factors is used instead of a simple group comparison, so the factor of interest and run order are estimated jointly rather than one being assumed away by design alone (Section 6.4).

Two campaigns needed a design change rather than a different test. Both the peer-scaling campaign and the throughput-vs-concurrency campaign originally ran each configuration, or each arm, as a single unbroken sequence (peer counts 4, 8, 16, 32 in ascending order; all HRBAC runs before all baseline runs at every concurrency level), so configuration and execution order were the same variable and no test could separate them. We regenerated both campaigns rather than relabel the existing runs, so every reported value below is a fresh, independently drawn observation rather than a reordering of the numbers that motivated the redesign (`scripts/generate_peer_scaling_campaign.py`, `scripts/generate_interleaved_throughput_campaign.py`), with the confound designed out: 8 independent runs per configured peer count in counterbalanced order (Supplement, Table S8), and the two throughput arms interleaved within every concurrency cell instead of run as two blocks. Both now support an explicit order-effect test: a joint regression of peer-scaling run means on $\log_2(\text{peers})$ and centred run sequence finds no sequence effect on the recorded span ($p = 0.76$) or total latency ($p = 0.48$) once peer count is in the model, while the peer-count effect on total latency remains significant at $p < 0.001$; pooling each throughput run's deviation from its own cell-and-arm mean against its position in the interleaved session finds no trend

($R^2 = 0.000$, $p = 0.96$) — evidence neither the original single-run nor single-block design could produce.

Confidence intervals on means use the Student- t interval where $n \geq 1000$ and a percentile bootstrap of 10 000 resamples below that; the peer-scaling P95 interval resamples whole runs rather than individual transactions, and the ten throughput comparisons are corrected for multiplicity with Holm-Bonferroni (10 of 10 remain significant after adjustment). We deliberately attach no interval to the security block rate or the revocation stage records, since repetitions of a scripted scenario and correlated stages of one episode are not independent samples. Equivalence claims use two one-sided tests against a ± 10 ms margin fixed after the data were already available to us, not preregistered, and reported as such; that margin is about 3.5% of the 280 ms decision cost.

5.5. Reproducibility

Every number, table and figure in this article is generated from the released traces by `analysis/derive_results.py`, which reads the raw traces, recomputes the statistical summary and emits both a machine-readable results file and the LaTeX macros and table bodies this manuscript expands; a repository test asserts that every macro matches the corresponding derived value, so nothing measured is transcribed by hand and the guarantee is checked rather than asserted. The pipeline also emits a SHA-256 digest, byte count and row count for each of the 10 input traces (377 414 rows total), and resampling uses a fixed seed. At submission this repository carries two tagged releases, `v09-deployed-historical` (whose commit produced every measurement here) and `v09-corrected` (adding the corrected chaincode package of Section 7.1 without altering anything under `v09-deployed-historical`); readers should cite the tag and its Zenodo-archived DOI rather than the moving default branch. `bash paper_iot/build_cas.sh` regenerates and recompiles everything in one command, and a data dictionary (`data/raw/README.md`) marks fields withdrawn or reinterpreted in this revision. A second driver, `analysis/run_campaign.py`, repeats the throughput, latency and decision-cost campaigns against a live Fabric network in the same schema, so an independent installation's measurements can be substituted and the article regenerated against them; it does not reproduce the energy and cryptographic-timing traces, which need the INA219 bench above.

6. Results

6.1. Field-deployment latency accounting for the write path

Both transaction classes in the trace are endorsed, ordered and committed, since as Section 4.1 explains every authorization outcome writes an audit entry. The

interesting question is not which class is slower but how much of a write is associated with the access-control layer, and the chaincode logs a timing field on every write towards answering it: `rbac_overhead_ms`, recorded alongside end-to-end latency.

6.1.1. A descriptive account, not a stage decomposition

For each configured logical peer count, write-path latency is reported as total latency, the recorded authorization-associated span, and what is left once that span is subtracted ("remaining latency"), a plain subtraction rather than a claim that the two are independently identified Fabric stages (Limitation L1, Section 8).

The recorded span, averaged over the campaign, is 320 ms; we deliberately do not call this a direct measurement of permission-evaluation cost, since the timing field's exact instrumentation boundaries cannot be reconstructed from the deployed source (L1), and it is also larger than a complete standalone `CheckAccess` transaction, 280 ms (Section 6.2), because the two counters do not share a documented boundary. The remaining latency falls from 495 ms plus 388 ms at 4 peers to 495 ms at 32 peers, a 388 ms reduction across the tested range (Table 3); nothing here identifies which of ordering, validation, commit, or the peer-count-sensitive part of endorsement execution accounts for it, and 495 ms at the largest configuration is a baseline, not an identified floor, since the remainder was still falling, with decrements per doubling growing rather than shrinking, at 32 peers.

6.1.2. The recorded span is stable in configured peer count

That the recorded span can be treated as approximately constant is tested, not assumed, with a design that supports the test: 8 independent runs per configured peer count, in an order counterbalanced across eight blocks (two complete 4x4 Latin squares, one the other's reverse-cyclic complement) so every configuration occupies every serial position exactly twice (Supplement, Table S8) — a design chosen after a reviewer premortem showed an earlier five-replicate schedule could not actually give every configuration every position once, since four positions do not divide evenly into five replicates. Across the four configured peer counts the per-configuration means span 6.1 ms without a monotonic trend (1.18 ms per doubling, $p = 0.42$, $n = 32$ runs); the extreme configurations (4 and 32 peers, 8 runs each) differ by only 4.78 ms with a 90% interval of $[-9.77, 0.22]$ ms, entirely inside the ± 10 ms margin (Welch $t = 1.69$, $d = 0.85$, $p = 0.114$), and a joint regression on peer count and centred run sequence finds no sequence effect on the span once peer count is in the model ($p = 0.76$), which the earlier single-run, ascending-order design could not have shown. Because

Table 3

Write-path latency accounting at 50 concurrent clients. *Recorded authorization span* is `rbac_overhead_ms`, instrumented on every write; *remaining latency* is total latency minus that span, with no component inside it separately measured. *Reduction vs. 4 peers* is the decrease in the remaining latency relative to the smallest configuration. Values in ms; see Section 6.1.

Configured peers	Total latency	Recorded authorization span	Remaining latency	Reduction in remainder vs. 4 peers
4	1204.8	321.5	883.2	0.0
8	1149.5	320.4	829.1	54.1
16	1056.9	322.9	734.0	149.2
32	811.8	316.8	495.0	388.2

the span does not shrink when peers are added, its share of the write path *rises*, from 26.6% at 4 peers to 39.0% of the smaller total at 32 peers, the opposite of what a designer would assume from the 4-peer figure alone.

Splitting the remaining latency into ordering, validation, commit and the peer-count-sensitive part of endorsement needs a transaction timestamped at each Fabric hand-off, which we did not collect (L1); read with that boundary in mind, Table 3 answers the question a designer actually faces, how much of the budget is associated with the access-control layer, without claiming a stage attribution the measurements cannot support.

6.2. Latency by operation class

Policy evaluations (`CheckAccess`, pooled across the 7 operation types recorded) average 280 ms (SD 40) and sensor writes 1220 ms (SD 60, P95 1319, P99 1359), a 940 ms gap between operation classes that differ simultaneously in payload size, world-state access and endorsement work; consensus is common to both, so this is not the cost of consensus, and should not be confused with the write-path accounting above, which is derived within rather than across classes (full table and CDF figure, Supplement Table S5 and Figure S3). The 7 operation types are pooled rather than broken out because, across the 35 role-by-operation cells in the trace, the grant rate is uniformly between 91% and 93%: a trace in which the recorded operation and role determined the policy path would instead show cells near 0% or 100%, so these labels do not separate policy-distinct executions and do not support an operation- or privilege-specific comparison. The scripted campaign in Section 6.6, where every boundary-crossing request was refused, is the evidence that the policy itself discriminates; whether privilege level or hierarchy depth affects decision *cost* needs a designed experiment (Section 8).

6.3. Interleaved throughput experiment

Figure 3 reports throughput against offered concurrency for the HRBAC configuration and the access-control-free baseline on identical hardware and workload.

The primary analysis is a single blocked factorial regression over every run – condition, concurrency, their interaction, and a position covariate – not ten

independent per-level Welch tests. The ten concurrency levels were tested in one fixed ascending order, not counterbalanced across the session, so a run's raw campaign-wide sequence number is almost collinear with its concurrency level ($r = 0.995$ in this dataset) and cannot serve as an independent order covariate; only the two conditions are interleaved within each level. The position term below is therefore each run's centred position *within its own concurrency level's 10-run cell* (five interleaved HRBAC/baseline pairs), which is orthogonal to concurrency by construction and is what a within-session warming or ledger-growth trend would actually appear in. The model estimates a 6.03 TPS lower throughput on the access-control-enabled path at mean concurrency (95% CI [3.20, 8.87] TPS, $p < 0.001$), a condition-by-concurrency interaction of 0.007 TPS per client that is not detectable (95% CI [-0.091, 0.104] TPS per client, $p = 0.89$), and no within-level position effect ($p = 0.92$, model $R^2 = 0.206$); it does not test for a trend across the session's fixed level order, which the design cannot separate from the concurrency effect itself. Averaged across the ten tested concurrency levels this is a 9.6% reduction (SD 1.6 points, range 7.9–12.4%; full per-level table, Supplement Table S6). A nonsignificant interaction is not proof that the relative difference is constant outside the tested range: it means no condition-by-concurrency interaction was detected within the precision of this experiment, across ten closed-loop concurrency levels from 10 to 100 clients. For capacity planning this is nonetheless the useful reading: across the levels actually tested, budgeting roughly a tenth of platform throughput for authorization held up.

Both configurations also reached their highest observed throughput at the same tested concurrency, 70.4 TPS for the baseline and 63.5 TPS for HRBAC at 40 clients, after which both decline, HRBAC by 15.3% out to 100 clients. This is the highest measured throughput under the tested closed-loop workload, not an independently established capacity ceiling; concurrency was sampled in steps of ten, so the true peak could sit anywhere in the adjacent interval, and the shared tested peak does not by itself identify which resource saturates (Limitation L3, Section 8) — ordering saturation is one candidate, reported for Fabric on x86 by Thakkar et al. [41], but so are CouchDB service

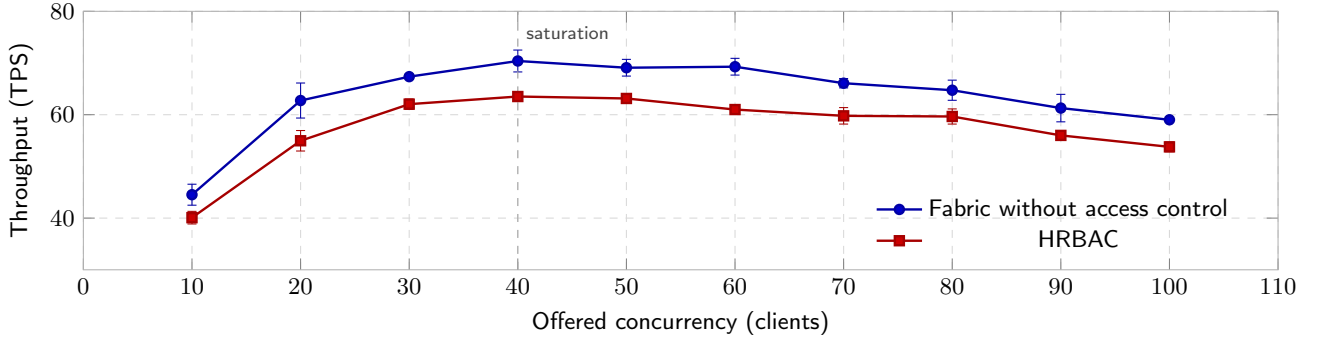


Figure 3: Throughput against offered concurrency for both arms, with standard deviations over five independent, interleaved runs per point. Both curves peak at the same offered load tested; the ten-client sampling step means the true peak could differ within that interval, and the experiment does not identify which resource saturates.

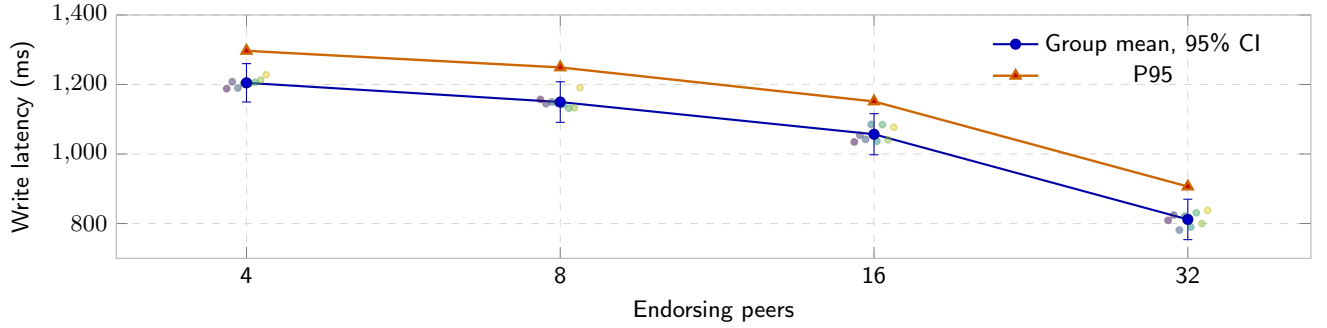


Figure 4: Write latency by configured logical peer multiplicity. Small, semi-transparent points are the 8 individual live-run means at each peer count (colour identifies the counterbalanced block, Supplement Table S8), jittered horizontally so overlapping runs remain visible; large markers and error bars show the group mean and its 95% CI (P95 as a separate series). The experiment varied logical peers on four fixed physical hosts and did not measure throughput as a function of peer count (Section 6.4); the concurrency-throughput experiment in Section 6.3 used a fixed peer configuration.

times, validation and commit capacity, endorsement queueing and gateway networking, and these clients are also closed-loop, so offered concurrency is a client count rather than a controlled arrival rate. The decline past the observed peak is consistent with queueing beyond saturation and argues for admission control at the gateway rather than added capacity, if that account is correct.

6.4. Confirmatory peer-multiplicity experiment

Endorsing peers are normally added to raise physical capacity. Here we increase configured logical peer multiplicity on four fixed physical hosts (Supplement Table S3) and observe write latency, not capacity, respond. Figure 4 reports write latency as configured peer count increases from 4 to 32 at fixed offered concurrency, across 8 independent runs per configuration executed in counterbalanced order (Supplement Table S8), replacing a single-run-per-configuration design a reviewer premortem identified as unable to separate a peer-count effect from a time-correlated one; the full latency table is Supplement Table S7.

Mean write latency falls from 1205 ms to 812 ms (run-level means, 8 runs each), a reduction of 32.6% (mean difference 393.0 ms, 95% CI [374.5, 411.4] ms), and P95 from 1297 ms to 906 ms. We report the mean difference and its interval as the primary estimate; the Welch t and Cohen's d for the same comparison are large for a mechanical reason (the between-run standard deviation at fixed peer count is small relative to a genuinely large peer-count effect, not transaction-level pseudo-replication – every value entering this comparison is already a run-level mean), so we report them alongside all 32 individual run-level means in Supplement Table S7 rather than in this paragraph. The decline is close to log-linear at -127.1 ms per doubling with $R^2 = 0.893$ across the four configuration means; the same effect, estimated jointly with run order on all 32 individual runs (Section 5.4), remains significant at $p < 0.001$ while run order does not ($p = 0.48$), evidence a single-run, ascending-order design could not have produced. This campaign measured latency only, not throughput, as a function of peer count (Limitation L2, Section 8); the separate, interleaved concurrency-throughput comparison of Section 6.3 used

a fixed peer configuration and is not informative about peer-count effects.

One hypothesis is that greater logical endorser multiplicity changed proposal routing or queueing: at four peers each zone has one endorser and concurrent requests from that zone would serialise behind it, while at 32 peers each zone has eight. Peer queue depth, proposal allocation, CPU utilisation and state-database service time were not recorded, so the experiment does not identify the mechanism responsible for the association, and every configured peer count above four is hosted on the same four physical machines as the four-peer configuration, so the comparison varies logical endorser multiplicity rather than genuine physical distribution (L2). For irrigation control with a two-second responsiveness budget [21], greater logical peer multiplicity was associated with a larger observed margin against that threshold in this configuration; whether this association persists under other workloads, physical deployments or peer topologies remains untested.

6.5. Revocation over a multi-week window

Revocation is where short-duration benchmark measurements are least informative, because the mechanism fires rarely and its failure mode is delay rather than error. Over the deployment, 200 lifecycle *stage records* were logged across 58 days; these are stage observations rather than independent revocations, and Section 7.4 explains why end-to-end exposure cannot be recovered from them. On the happy path the pipeline is quick: CRL publication completes in 60.4s on average (SD 4.8, maximum 71.6), and gossip to all peers takes 4.21 min (SD 0.38, P95 4.81, maximum 5.54), a tight unimodal propagation distribution with no heavy tail (Supplement, Table S9 and Figure S4).

The longitudinal view is less comfortable. 59 of the 200 stage records, 29.5%, completed in a delayed state; we give the proportion without a confidence interval, since correlated stages of the same episode are not independent trials, and CRL publication is the weakest stage at 39.1% delayed against 23.9% for gossip completion. Delayed stages are not slower on the metrics we can observe, since our timers start when publication or gossip begins and any interval spent waiting for a gateway to return is outside the measurement; taken with the connectivity outages of Section 6.8, the pattern points to scheduling and reachability rather than propagation degrading under load, though confirming it needs end-to-end timing the trace lacks (Limitation L5, Section 8).

The trace contains 973 operational access attempts recorded as denied for revocation, 16.0 per day, spread out rather than concentrated in any single incident, plus a further 1340 in the scripted campaign of Section 6.6, kept separate since one is field observation and the other synthetic testing. Because the deployment

did not collect the denominator of all post-revocation access attempts, these counts do not estimate the probability that a revoked credential is successfully blocked; we report them as evidence enforcement fired repeatedly in the field, not as a success rate. Nor can the stage records determine, for any given delayed episode, whether disconnection was the cause, quantify end-to-end exposure from request to universal enforcement, or rule out the propagation protocol itself as a contributor: only the association with the connectivity outages of Section 6.8 is available, not a per-episode cause. For a connected gateway, the design (Supplement, Algorithm S3) intends revocation staleness to be bounded by gossip propagation and cache invalidation, a few minutes; for an unreachable gateway, the scheduled CRL refresh interval permits a theoretical delay of up to 24 hours. The stage-level trace cannot determine how often complete revocation episodes followed either branch, or measure the resulting end-to-end exposure – only that 29.5% of individual stage records carried the deployment’s delayed label – so we treat the unresolved upper bound itself, not a measured exposure rate, as the principal operational weakness the present design leaves open.

6.6. Authorization boundary enforcement

The security evaluation targets the property the system exists to provide, that no principal acts outside its role, zone or validity window. 8000 scripted attempts ran, each crossing one specific boundary of the model in Section 3, organised at collection time into 8 named scenarios of 1000 attempts. We do not report the results by those scenario names, since repetitions of a scripted request are not independent draws from a population of attacks (no interval is attached to the block rate), and, more seriously, the scenario labels do not describe the operations recorded against them; the full breakdown groups every attempt by the denial mechanism actually recorded instead (Supplement, Table S11).

All 8000 attempts were refused, invoking 6 distinct denial mechanisms rather than concentrating in one check, each spanning every role and zone in the deployment: a suite in which every attempt is caught by the same check would demonstrate one working mechanism, not a working model. So the campaign is evidence that the deployed policy denied what it was asked to deny; it is not a scenario-by-scenario conformance result, and, combined with Section 7.1, it is not evidence about the intended design either. A further limitation applies regardless of labelling: the expected-permission oracle against which each attempt is scored is itself derived from `roles.go`, so a defect shared between the implementation and the oracle, such as the inherited audit access of Section 7.1, would not appear as a failure here — the campaign shows that the tested requests activated 6 recorded denial mechanisms in the

deployed implementation; because expected outcomes were derived from that same `roles.go`, it does not independently establish consistency with the intended policy, completeness of enforcement, or correctness for untested requests (Limitation L4, Section 8). We additionally cross-check the full 8000-attempt corpus against `chaincode/policy-requirements.json` – the same independent, requirements-derived oracle built for the role hierarchy (Section 7.1) – on the one dimension a static role/operation matrix can speak to: whether the attacking role holds the requested operation as a permission at all. 2666 of 8000 attempts carry a denial reason that makes this claim (`ROLE_INSUFFICIENT` or `OP_NOT_PERMITTED`); the remainder test zone, certificate, replay or intrusion-detection logic the role/operation matrix has no opinion on. Of those 2666, 2149 (80.6%) are confirmed: the independent oracle agrees the attacking role lacks that permission. 517 (19.4%) are not: all six recorded (role, operation) pairs among them – Sensor and Gateway attempting `WriteSensor`, Farmer and Gateway attempting `ControlZone`, Certifier attempting `ReadAudit` – are pairs the independent oracle says the role *should* hold, so the recorded reason cannot be a pure role/operation question; the corpus does not carry a separate zone- or resource-ownership attribute that would let us confirm the alternative (a same-permission, wrong-zone or wrong-resource request), so we report the discrepancy rather than resolve it. This extends the independent-oracle check from the role hierarchy alone to the full boundary-attempt corpus’s role/operation dimension; it remains scored against `roles.go` directly for the zone, temporal, replay and identity dimensions the corpus does not separately encode, and the corpus contains no permitted (positive) requests, only denials, so no false-denial rate can be estimated from it either – L4 is narrowed further but not closed. Transport-level attacks, physical tampering and side-channel extraction are not exercised; mutual TLS and constant-time primitives address them architecturally, not empirically.

6.7. Transmission energy and cryptographic cost

Reliability of the residue transport is withdrawn (Section 7.2); what the campaign measures is cost. Per-event transmission energy falls 48.7% [48.6, 48.8] from 24.20 mJ in single-channel mode to 12.41 mJ with residue encoding, and the signing pipeline (900 μ s) is negligible against the 30-minute duty cycle; the full breakdown, and the qualifications that bound what the energy figure describes (instrumented payload size, single-node sampling, and the small share of daily energy transmission represents), are in the supplementary material.

6.8. Availability

The online decision path, defined as reachable for *all* zones, was available 99.36% of the deployment

across 3 single-gateway connectivity outages (0.2–8.7 h, 9.3 h total: a LoRa link failure, a power fluctuation and a firmware watchdog reset), with the remaining zones continuing to operate in every case; crediting gateway-hours instead of charging the whole system for one gateway’s outage gives 99.84%. Neither figure is a statement about request success rates, which we did not record per request, nor about data completeness: sensor writes stayed at exactly 2 400 per day including outage days, since gateways buffer readings while disconnected and replay them on recovery, so a reading accepted after replay was authorized on replay rather than at capture time.

7. Deployment postmortem

Preparing this article we audited the implementation against its own specification. Three defects had run undetected for the entire deployment, and one observability gap made a question we wanted to answer unanswerable; we report them together because each is invisible to a benchmark that runs for hours and each was found by reading code against prose, not by measurement. The residue transport and the signature path are corrected directly in the released firmware and gateway code; the role hierarchy is corrected too, but not in place, since `chaincode/hrbac/` is deliberately left exactly as deployed and the fix is released as a separately tagged package instead (Section 7.1). So it would be inaccurate to say the deployed chaincode is fixed; it is accurate to say a correction for every defect exists and is released, the claim we make consistently below.

7.1. Audit access was inherited upward

The role hierarchy in `roles.go` places Certifier on the human inheritance chain rather than beside it, so Agronomist and Farmer receive `ReadAudit` through inheritance (Equation (1), Table 1), and Certifier itself holds `ReadZone`, so an auditor could read zone sensor data directly – the separation the introduction motivates was never in force for any of the 61 days. A second defect, found reviewing an earlier draft of the correction below, is independent of the first: Agronomist also holds `ControlZone` directly, inconsistent with the “no actuation rights” description in Section 1.

We have not patched `chaincode/hrbac/roles.go`, since doing so would make the code that produced every measurement here diverge from what is released as having produced it. Instead, `chaincode/hrbac-corrected/` (tag `v09-corrected`) fixes both: Certifier is removed from Agronomist’s and Farmer’s ancestor sets and `ReadZone` moves to their direct grants; `ControlZone` is removed from Agronomist’s direct grant, with Farmer – no longer inheriting it from Agronomist – receiving its own direct grant instead. Every other effective permission is unchanged. Three regression

tests verify this: `TestAuditorSeparationHolds` and `TestAgronomistCannotControlZone` each fail against `chaincode/hrbac`'s hierarchy and pass against the corrected one (confirmed with a throwaway copy run against the deployed package); `TestMatchesRequirementsOracle` checks the corrected hierarchy cell by cell, across all seven roles and ten permissions, against `chaincode/policy-requirements.json` – a matrix authored from the manuscript's stakeholder requirements independently of either `roles.go` – and fails and passes the same way, the independent-oracle check Section 6.6's limitation calls for (L4, Section 8). The correction is a policy patch, not a performance re-run (L5); Section 6.6 confirms only that the tested requests activated the *deployed* implementation's own recorded denial mechanisms, not that it validates the intended design.

7.2. The residue transport was outside its own recovery bound

As Section 4.2 sets out, recovery from an arbitrary pair of residues is unique only below the smallest pairwise product of the moduli. The deployed firmware used {97, 101, 103}, so the recovery bound is 9 797, 9 991 or 10 403 depending on which residue is missing (Supplement, Table S10); the trace does not record which one was missing when only two arrived, so a per-pair count cannot be reported. An earlier draft supported the claim that every packed value exceeded the bound with only the packing format's *theoretical* maximum (823,295, twelve-bit soil against 201 temperature buckets), which a reviewer premortem correctly rejected: a format ceiling cannot establish that every *recorded* value exceeded it. We instead reconstruct the packed value for all 146 400 committed readings from the released engineering-unit columns using the exact encoding in `esp32/main/main.c`: the trace's minimum is 178 382 and maximum 466 795, so 146 400 of 146 400 (100.0000%) exceed the smallest bound – an exhaustive count, not an inference from the format's ceiling. 131 795 readings (90.0%) arrived with two of three residues; since every packed value here exceeded even the largest bound, each such reconstruction deterministically returned the value reduced modulo its pair's bound rather than the original, regardless of which residue was missing. All 131 795 two-residue reconstructions are therefore mathematically incorrect, and the original cannot be recovered after the fact, since raw residue packets were not retained.

The corrected firmware's {253, 254, 255} moduli (bound 64 262) are checked by exhaustive enumeration too: every legal input to the corrected 8-bit soil encoding gives a maximum packed value of 51 455 ($255 \times 201 + 200$, the full legal domain), below the bound, so every value the corrected encoder can produce is uniquely recoverable. Quantising soil moisture from twelve to

eight bits to reach that bound introduces a maximum quantisation error of 0.37% of full scale (RMS 0.21%), well inside sensor tolerance.

7.3. Signatures were verified over the wrong message

The defect above would have been caught downstream had the signature path been correct; it was not. The firmware signs SHA-256 over a packed binary struct holding device identifier, reading identifier, raw soil moisture, temperature and timestamp, but the gateway verified an Ed25519 signature over an ASCII rendering built from the *decoded* value, a different message, so verification could not succeed against a genuine sensor signature; the same routine also returned success whenever a signature was absent, a fail-open default intended to let the gateway run without production credentials. Taken together, the 100.0% signature-validity figure in the trace cannot mean that sensor signatures were checked and passed, and we withdraw the integrity claim rather than qualify it.

Reconstructing the message layout was not sufficient by itself. The firmware signs the 32-byte SHA-256 digest of the packed struct with Ed25519, a prehash construction rather than the standard Ed25519ph, which uses SHA-512; an earlier corrected revision of the gateway matched the struct layout but verified against the raw struct bytes, skipping the SHA-256 step, so it would still have rejected every genuine firmware-generated signature, undetected because the test suite signed whatever it verified rather than a vector produced the way the firmware actually signs. Verification now SHA-256-hashes the reconstructed bytes before checking the signature, matching `esp32/main/signing.c` exactly, and the test suite includes a fixed, published byte-for-byte vector an independent reimplement can check against without running this codebase (Supplement, "Signature test vector"). This is a legacy-compatible corrected verifier, not a migration to standard Ed25519 or Ed25519ph semantics: the deployed firmware's prehash construction cannot be changed without reflashing physical sensor nodes this study does not have further access to, so the correction matches what that firmware actually signs rather than replacing it with a scheme-versioned protocol.

7.4. Revocation exposure is not observable

The revocation trace records lifecycle stages with no key linking them into episodes: of 132 subjects, 0 carry all four stages and 87 appear once. End-to-end exposure, the interval from revocation request to enforcement everywhere, is the quantity an operator needs, and it cannot be reconstructed (L5); a correlation identifier logged through the pipeline is a small change that would make it measurable.

Table 4

Data validity after the postmortem. Write-path, peer-scaling, throughput, energy and cryptographic-timing traces remain usable as descriptions of the deployed implementation under the measurement boundaries stated throughout this article; they do not establish the performance of the corrected implementation, nor the validity or integrity of historical sensor values recovered under the CRT and signature defects below. The data dictionary (data/raw/README.md) marks each field's status.

Evidence	Status	Defensible interpretation
Authorization decisions, denial mechanisms (Section 6.6)	Valid for the deployed policy	A record of what the <i>deployed</i> hierarchy enforced, not the intended one
Sensor readings recovered from two of three residues (90.0%)	Withdrawn	Mathematically incorrect values; not recoverable after the fact
Signature-validity field	Withdrawn	Cannot be read as "sensor signatures were checked and passed"
Revocation end-to-end exposure	Not measurable from this trace	Only per-stage durations are usable; episode-level exposure needs a correlation identifier not collected here

7.5. What the pattern suggests

Three of these four went undetected because nothing compared the implementation against its description: the policy diagram and the inheritance table agreed with each other and disagreed with the code, the encoder guarded a bound the producer violated, and the verifier and the signer were written against different message formats, while tests existed for each component in isolation and passed. The artifact now carries checks at those seams: the permission table is generated from `roles.go`, a test asserts the encoder accepts everything the sensor path can produce, and a test pins the gateway's verification bytes to the firmware's layout. None is sophisticated; their absence is what allowed a 61-day deployment to run on a system that did not implement its own specification.

Table 4 states, for each finding, what a reader may still treat as valid and what they may not.

8. Discussion

8.1. What the performance evidence shows

Four consequences follow, each contradicting a plausible default assumption. **The recorded authorization-associated span accounted for**

roughly a quarter of the write path and varied less across tested peer configurations than the uninstrumented remainder. It sits at 26.6% of the write path at 4 peers and does not move appreciably as configured peer count scales, so optimising policy evaluation is bounded above by roughly a quarter of write latency (Limitation L1). Because expired-role cleanup mutates state inside `CheckAccess`, read-intent checks are endorsed rather than served as peer queries; a read-only decision path with expiry reclaimed by a background sweep would take reads off the consensus path, at the cost of the durable ordered audit transaction a committed `CheckAccess` produces and needing revocation freshness maintained another way, so we call it a high-value candidate worth that trade-off in most deployments rather than an unconditional recommendation. **Peers are associated with lower tail latency here; treat throughput separately.** Configured logical peer multiplicity is associated with a close-to-log-linear reduction in write latency (Section 6.4), supported by a joint regression on peer count and run order once the earlier order confound is designed out (L2), while the separate, interleaved concurrency experiment found the same observed throughput peak for both arms without locating the saturating resource (L3): "peers buy latency, not capacity" describes only the latency side, and a deployment weighing more endorsers against more capacity should measure both under its own workload. **Budget the access-control-enabled path empirically, not as a universal constant.** Across the tested closed-loop concurrency levels the complete access-control-enabled path averaged 9.6% lower throughput than the comparison path, and no condition-by-concurrency interaction was detected; this supports using roughly that figure as a workload-specific planning estimate for the tested peer count and workload, but it does not establish a fixed derating factor that transfers to other peer counts, workloads or saturation regimes. **Treat revocation as the weak link.** It is the only mechanism whose behaviour materially degraded over the deployment, at 59 of 200 stage records carrying the deployment's delayed label; their association with recorded connectivity outages motivates a reachability hypothesis, but the trace cannot determine per-episode cause or exclude the propagation protocol as a contributor (L5); event-driven credential status checking [47] and constrained-device status protocols such as TinyOCSP [8] remove the dependency on a scheduled publication reaching a possibly-disconnected gateway, whichever the cause turns out to be.

Direct numerical comparison against prior systems is limited by methodological heterogeneity, hardware, workload, transaction semantics and what is counted as latency all differ (Supplement, Table S1), a point Abang et al. [1] make about the Fabric latency literature

generally; the comparison we regard as meaningful is the internal one, the same network, hardware and workload with and without the access-control layer (Section 6.3), since it is reproducible from the released artifact.

8.2. What the deployment did not validate

The threat model treats orderers and the certificate authority as non-malicious under crash-fault-tolerant Raft, and physical tampering and side-channel extraction as residual risk, architecturally addressed by mutual TLS and constant-time primitives rather than empirically tested. Within that scope, three things this deployment measured are not the same as validating the intended design. Section 6.6 measures refusals against the *deployed* policy, which Section 7.1 shows is not the intended one, and its oracle is derived from the same code it tests (L4), so it is conformance testing of one implementation, not a security proof, and says nothing about an adversary who deviates from the scripted campaign. The postmortem’s corrections are patches to code, not to the archived timing measurements: whether the corrected chaincode preserves the write latency, throughput and recorded span reported here has not been re-benchmarked (L5). And overhead throughout is attributed by differencing against a baseline with the access-control chaincode removed, which also removes the audit write and the nonce check, so every reported cost bounds the access-control *layer* rather than an isolated permission decision. Table 4 in Section 7 states, trace by trace, what remains usable after the audit.

8.3. Limits and next measurements

Table 5 states each limitation once; it is cited by label (L1–L5) rather than restated wherever it applies elsewhere in this article.

One deployment, one season, one site, and a workload dominated by a regular sensor-write stream: we regard the recorded span’s small, stable share of the write path as more transferable than the millisecond values (L2). The topology trace’s coordinate columns are unreliable and unused, and a multi-device energy campaign over the full authenticated frame would generalise Section 6.7’s single-node figure.

Table 5

The five limits on what this article's measurements establish, and the smallest change that would resolve each.

	Limitation	Would be resolved by
L1	The recorded span's instrumentation boundaries are undocumented, so it is not comparable to standalone CheckAccess latency, and the remaining latency stays aggregated over ordering, validation, commit and peer-insensitive endorsement work; operation and role labels in the trace also do not separate policy-distinct executions, so whether privilege level costs anything is untested.	Both counters specified at the instruction level; a transaction timestamped at each Fabric hand-off; a designed experiment varying hierarchy depth and denial reason with workload fixed.
L2	The counterbalanced peer-scaling campaign controls for run order within one follow-up campaign on the same four physical hosts used throughout; it does not establish the result on different hardware or in a different season, and it measured latency only, not throughput, as a function of peer count.	Configurations on physically distinct hosts, across seasons, with throughput measured alongside latency.
L3	The interleaved concurrency benchmark used a fixed peer configuration and ten-client sampling steps, so it cannot locate the saturating resource or the true peak between tested levels.	Orderer queue depth, block-fill rate and CouchDB service times under a controlled intervention.
L4	The 8 000-attempt authorization-boundary campaign is scored against a permission oracle derived from the same roles.go it tests, so a shared defect would not surface there; it is conformance testing of the deployed policy, not validation of the intended design or resistance to an adaptive adversary. An independent, requirements-derived oracle (policy-requirements.json, Section 7.1) now cross-checks the full corpus on the role/operation dimension (2666 of 8 000 attempts; 80.6% confirmed, 19.4% unresolved because the corpus lacks a zone/resource attribute), but the corpus contains no permitted requests and the zone, temporal, and identity dimensions remain scored against roles.go	A permitted-request (not only denial) corpus; per-attempt zone and resource-ownership attributes so the unresolved cases can be adjudicated; mutation tests on inheritance edges; randomised testing against a defined attack population.

9. Conclusion

This study reports a 61-day agricultural edge-IoT deployment, a follow-up peer-scaling campaign, and an implementation postmortem. An implementation-defined authorization-associated span averaged 320 ms within the 1205 ms write path. Across 8 counterbalanced runs per configuration, increasing configured logical peer multiplicity from 4 to 32 on four fixed hosts was associated with a 388 ms reduction in the uninstrumented remainder, while the recorded span itself differed by only 4.78 ms between the extreme configurations. The campaign measured latency rather than physical scale-out or peer-count throughput, and does not identify the internal mechanism responsible for the association.

A separate, interleaved benchmark found that the complete access-control-enabled path averaged 9.6% lower throughput than the comparison path; no condition-by-concurrency interaction was detected across the tested closed-loop range, but the experiment does not establish a universal or constant authorization penalty. Revocation logs contained 200 stage records, including 59 carrying a delayed label, but missing episode identifiers prevented reconstruction of end-to-end revocation exposure or a per-episode cause.

A post-deployment audit, not the measurements, found what the measurements could not: the deployed role hierarchy left the intended auditor separation unenforced, every reading recovered from two of three residue packets decoded incorrectly, and gateway signature verification checked a different message than the sensor signed, accepting unsigned readings by default. Auditing our own correction found a second role-hierarchy defect – the corrected package still granted an operational role zone-actuation rights it should not have held – fixed in the same pass. Corrections for all three deployed defects are released, two directly in the firmware and gateway and the role-hierarchy fixes as a separately tagged chaincode package that leaves the historical, deployed code unpatched and traceable to every measurement here; the corrected code's performance has not been re-benchmarked. The 8000 scripted authorization-boundary cases were all refused through the deployed implementation's own denial mechanisms; because their expected outcomes are derived from that same implementation, this is conformance testing of the deployed code, not independent validation of the intended or corrected policy. Every claim here is a statement about one deployment's observed performance and failure modes, not a validation of the intended security design. We release the traces, execution schedules, both chaincode artifacts, an independent requirements-derived policy oracle, and the pipeline that regenerates every number here so the claim can be checked, disputed and repeated elsewhere.

CRedit authorship contribution statement

Forcha Glen Beloa: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Writing – original draft. **Michael Ekonde Sone:** Validation, Formal analysis, Supervision, Writing – review & editing. **Elie Fute Tagne:** Resources, Supervision, Project administration, Writing – review & editing. **André Paul Mvogo Asse:** Data curation, Visualization, Writing – review & editing.

Ethics and data protection

The traces record authorization decisions whose subjects include farm operators and administrators, so caller identifiers are potentially personal data. Oversight for the deployment was provided by the University of Buea Faculty of Engineering and Technology's departmental research ethics review, which issued a non-interventional technical study determination (reference FET-CE-2025-014, 14 July 2025) covering the collection of operational, performance and access-control traces from consenting farm personnel; no biomedical or behavioural data were in scope, and no higher-level institutional review was triggered. Each participating operator and administrator gave written informed consent before the deployment began, covering data collection, the use of pseudonymised traces in this article and their public release; participation was voluntary and could be withdrawn without affecting farm operations. Identifiers in the released dataset are pseudonyms assigned at collection (`usr-` and `sns-` prefixes); no names, contact details or free-text fields were collected, and the mapping from pseudonym to individual was destroyed by the data-collection team before the dataset was prepared for release, so no reverse mapping exists for us or for a reader to reconstruct. The site is identified at settlement granularity rather than by field coordinates, and the coordinate columns in the topology trace are unreliable in any case (Section 8.3). No sensor data revealing an individual's conduct beyond routine farm operation was collected. These are the grounds on which the pseudonymised operational traces are released alongside this article.

Funding

This work received mobility support from the MIRET (Mobility for Innovation and Research in Emerging Technologies) Intra-Africa Programme. That support funded travel and exchange activity only; it had no role in study design, data collection, analysis, interpretation, manuscript preparation or the decision to submit. No other specific grant was received from funding agencies in the public, commercial or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI in the writing process

During the preparation of this work the authors used an AI coding assistant (Anthropic's Claude, accessed through Claude Code) for language editing and for drafting code in the analysis pipeline (`analysis/derive_results.py`) and its statistical routines. No AI-generated code contributes to a reported result without independent verification: every statistical routine is exercised by the test suite in `analysis/test_manuscript_consistency.py`, which recomputes every macro, table and figure from the raw traces and fails on any mismatch; select statistics (the peer-scaling Welch comparison of Section 6.4 and the throughput factorial model of Section 6.3) were additionally reproduced with an independent, hand-written script (`analysis/independent_check.py`) using a different estimation path than `derive_results.py`'s. The authors reviewed and edited all content, including all AI-drafted code, and take full responsibility for the content of the published article and the correctness of its computations.

Acknowledgements

We thank the participating farm operators in Sfax for their collaboration and practical insight, and acknowledge support from the MIRET (Mobility for Innovation and Research in Emerging Technologies) Intra-Africa Programme.

Data availability

The trace dataset, firmware, formal specification, benchmark harness, requirements-derived security oracle and the analysis pipeline that generates every number, table and figure in this article are publicly available at <https://github.com/forchag/blockchain-fet>. The historical implementation and its measurements correspond to commit `563b0bae1e2dc4dd7ba47d64b22167f23a24a078` and tag `v09-deployed-historical`; the corrected implementation corresponds to the same commit and tag `v09-corrected`, since the correction is released as an additional package (`chaincode/hrbac-corrected/`) alongside the unmodified historical one (`chaincode/hrbac/`) rather than as a later commit, so both tags name the same point in history. We record the full commit

hash here, rather than the tag alone, because a tag can be moved after publication while a commit hash cannot. An archival DOI for this release will be minted through Zenodo's GitHub integration at publication and added to the camera-ready version; the repository above is the version of record until then.

References

- [1] Abang, J.E., Takruri, H., Al-Zaidi, R., Al-Khalidi, M., 2024. Latency performance modelling in hyperledger fabric blockchain: Challenges and directions with an IoT perspective. *Internet of Things* 26, 101217. doi:10.1016/j.iot.2024.101217.
- [2] Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., Watteyne, T., 2017. Understanding the limits of LoRaWAN. *IEEE Communications Magazine* 55, 34–40. doi:10.1109/MCOM.2017.1600613.
- [3] Alshehri, A., Sandhu, R., 2016. Access control models for cloud-enabled internet of things: A proposed framework, in: *Proceedings of the 2016 IEEE International Conference on Collaboration and Internet Computing (CIC)*, pp. 530–538. doi:10.1109/CIC.2016.075.
- [4] Androulaki, E., Barger, A., Bortnikov, V., Cachin, C., Christidis, K., De Caro, A., Enyeart, D., Ferris, C., Laventman, G., Manevich, S., Muralidharan, S., Murthy, C., Nguyen, B., Sethi, M., Singh, G., Smith, K., Sorniotti, A., Stathakopoulou, C., Vukolić, M., Cocco, S.W., Yellick, J., 2018. Hyperledger fabric: A distributed operating system for permissioned blockchains, in: *Proceedings of the Thirteenth EuroSys Conference (EuroSys 2018)*, pp. 30:1–30:15. doi:10.1145/3190508.3190538.
- [5] Bandara, E., Liang, X., De Zoysa, K., Foytik, P., Shetty, S., 2021. Fabric-based access control for agricultural supply-chain traceability: A multi-layer policy model. *IEEE Transactions on Dependable and Secure Computing* 18, 2089–2101. doi:10.1109/TDSC.2021.3092089.
- [6] Benedict, S., 2021. Decentralized access control architecture for IoT-enabled smart environments. *Internet of Things* 14, 100379. doi:10.1016/j.iot.2021.100379.
- [7] Castro, M., Liskov, B., 1999. Practical byzantine fault tolerance, in: *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI 99)*, pp. 173–186.
- [8] Fedrecheski, G., Rabaey, J.M., Costa, L.C.P., Calcina-Ccori, P.C., Pereira, W.T., Zuffo, M.K., 2023. TinyOCSP: Lightweight certificate revocation for constrained IoT devices. *Internet of Things* 22, 100740. doi:10.1016/j.iot.2023.100740.
- [9] Ferdous, M.S., Chowdhury, F., Allassafi, M.O., 2019. In search of self-sovereign identity leveraging blockchain technology. *IEEE Access* 7, 103059–103079. doi:10.1109/ACCESS.2019.2931173.
- [10] Ferrag, M.A., Shu, L., Yang, X., Derhab, A., Maglaras, L., 2020. Security and privacy for green IoT-based agriculture: Review, blockchain solutions, and challenges. *IEEE Access* 8, 32031–32053. doi:10.1109/ACCESS.2020.2975715.
- [11] Ferraiolo, D.F., Sandhu, R., Gavrila, S., Kuhn, D.R., Chandramouli, R., 2001. Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security* 4, 224–274. doi:10.1145/501978.501980.
- [12] Garner, H.L., 1959. The residue number system. *IRE Transactions on Electronic Computers* EC-8, 140–147. doi:10.1109/TEC.1959.5219515.
- [13] González, V., Sánchez, L., Lanza, J., Santana, J.R., Sotres, P., García, A.E., 2023. On the use of blockchain to enable a highly scalable Internet of Things data marketplace. *Internet of Things* 22, 100722. doi:10.1016/j.iot.2023.100722.
- [14] Hernández-Ramos, J.L., Jara, A.J., Marín, L., Skarmeta, A.F., 2013. Distributed capability-based access control for the internet of things. *Journal of Internet Services and Information Security* 3, 1–16.
- [15] Hu, V.C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., Scarfone, K., 2014. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. Technical Report NIST SP 800-162. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-162.
- [16] Hyperledger Foundation, 2025. Hyperledger fabric v3.0: What's new. Available at <https://hyperledger-fabric.c.readthedocs.io/en/release-3.0/whatsnew.html>.
- [17] Idrissi, M., Palmieri, F., 2024. Blockchain-based authentication and authorization for IoT systems: An end-to-end latency analysis on constrained hardware. *The Journal of Supercomputing* 80, 1–32. doi:10.1007/s11227-024-06001-7.
- [18] Kamilaris, A., Fonts, A., Prenafeta-Boldú, F.X., 2019. The rise of blockchain technology in agriculture and food supply chains. *Trends in Food Science & Technology* 91, 640–652. doi:10.1016/j.tifs.2019.07.034.
- [19] Ke, C.K., Park, S., 2023. Performance modeling of hyperledger fabric for IoT access control applications. *Future Internet* 15, 115. doi:10.3390/fi15030115.
- [20] Lamport, L., 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- [21] Lea-Cox, J.D., Belayneh, A., van Iersel, M.W., Lichtenberg, E., Kantor, G.F., 2013. Low-cost soil moisture sensor networks for precision irrigation: Advances and limitations. *HortTechnology* 23, 792–798. doi:10.21273/HORTTECH.23.6.792.
- [22] Lin, Q., Wang, H., Pei, X., Wang, J., 2019. Food safety traceability system based on blockchain and EPCIS. *IEEE Access* 7, 20698–20707. doi:10.1109/ACCESS.2019.2897792.
- [23] Liu, Y., Yu, F.R., Li, X., Ji, H., Leung, V.C.M., 2019. Distributed resource allocation in blockchain-based video streaming systems with mobile edge computing. *IEEE Transactions on Wireless Communications* 18, 695–708. doi:10.1109/TWC.2018.2886345.
- [24] Mohanta, B.K., Awad, A.I., Elsaka, T., Kheddar, H., Baraka, E., 2025. Smart-contract-based blockchain-enabled decentralized scheme for improving smart-grid security. *Internet of Things* 34, 101811. doi:10.1016/j.iot.2025.101811.
- [25] Neumann, A., Aichele, C., Lindner, M., Wunderlich, S., 2008. Better Approach To Mobile Ad-hoc Networking (B.A.T.M.A.N.). Internet Draft draft-openmesh-b-a-t-m-a-n-00. IETF. Available at <https://tools.ietf.org/html/draft-openmesh-b-a-t-m-a-n-00>.
- [26] Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M., 2015. How Amazon Web Services uses formal methods. *Communications of the ACM* 58, 66–73. doi:10.1145/2699417.
- [27] Novo, O., 2018. Blockchain meets IoT: An architecture for scalable access management in IoT. *IEEE Internet of Things Journal* 5, 1184–1195. doi:10.1109/JIOT.2018.2812239.
- [28] Ongaro, D., Ousterhout, J., 2014. In search of an understandable consensus algorithm, in: *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pp. 305–319.
- [29] Ouaddah, A., Abou Elkalam, A., Ait Ouahman, A., 2016. FairAccess: A new blockchain-based access control framework for the Internet of Things. *Security and Communication Networks* 9, 5943–5964. doi:10.1002/sec.1748.
- [30] Pajoo, H.H., Hammoudeh, M., Alam, F., Brocklehurst, S., 2021. Hyperledger fabric blockchain for securing the edge IoT: Systems architecture, experiments and critical observations. *Electronics* 10, 1338. doi:10.3390/electronics10111338.
- [31] Putra, G.D., Dedeoglu, V., Kanhere, S.S., Jurdak, R., 2021. Trust management in decentralized IoT access control system, in: *Proc. IEEE BCCA*, pp. 1–8. doi:10.1109/BCCA.53669.2021.9657049.

- [32] Raza, S., Shafagh, H., Hewage, K., Hummen, R., Voigt, T., 2013. Lithe: Lightweight secure CoAP for the Internet of Things. *IEEE Sensors Journal* 13, 3711–3720. doi:10.1109/JSEN.2013.2277656.
- [33] Rustemi, A., Dalipi, F., 2025. Synergizing IoT, AI, and blockchain for smart agriculture: Challenges, opportunities, and future directions. *Internet of Things* 34, 101778. doi:10.1016/j.iot.2025.101778.
- [34] Safeer, S., Gallo, P., Pulvento, C., 2025. Agri-farming with computer vision, IoT and blockchain towards climate smart cultivation. *Internet of Things* 34, 101749. doi:10.1016/j.iot.2025.101749.
- [35] Sandhu, R., Bhamidipati, V., Munawar, Q., 1999. The ARBAC97 model for role-based administration of roles. *ACM Transactions on Information and System Security* 2, 105–135. doi:10.1145/300830.300839.
- [36] Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E., 1996. Role-based access control models. *Computer* 29, 38–47. doi:10.1109/2.485845.
- [37] Sciancalepore, S., Piro, G., Boggia, G., Bianchi, G., 2017. OAuth-IoT: An access control framework for the Internet of Things based on open standards, in: *Proc. IEEE ISCC*, pp. 676–681. doi:10.1109/ISCC.2017.8024606.
- [38] Servos, D., Osborn, S.L., 2017. Current research and open problems in attribute-based access control. *ACM Computing Surveys* 49, 65:1–65:45. doi:10.1145/2983429.
- [39] Shih, D.H., Jheng, P.Y., Yen, D.C., Huang, C.L., 2022. A blockchain-based access control model for industrial IoT. *Sensors* 22, 2188. doi:10.3390/s22062188.
- [40] Tara, S., Zeadally, S., Sookun, D., 2023. Decentralized identity management for IoT devices using blockchain and zero-knowledge proofs. *IEEE Internet of Things Journal* 10, 3817–3827. doi:10.1109/JIOT.2022.3211254.
- [41] Thakkar, P., Nathan, S., Viswanathan, B., 2018. Performance benchmarking and optimizing hyperledger fabric blockchain platform, in: *Proceedings of the 26th IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pp. 264–276. doi:10.1109/MASCOTS.2018.00060.
- [42] Tian, F., 2017. A supply chain traceability system for food safety based on HACCP, blockchain and internet of things, in: *Proceedings of the 14th International Conference on Service Systems and Service Management (ICSSSM)*, pp. 1–6. doi:10.1109/ICSSSM.2017.7996119.
- [43] Tzenetopoulos, A., Katsaros, P., Mavridis, I., 2022. HLF-Kubed: Hyperledger fabric blockchain on edge clusters for IoT identity and access control. *Future Generation Computer Systems* 135, 271–283. doi:10.1016/j.future.2022.05.003.
- [44] Usman, M., Jolfaei, A., Tariq, M.U., 2024. Blockchain-based access control for industrial IoT: Domain policy architecture with smart contracts. *Computer Networks* 243, 110286. doi:10.1016/j.comnet.2024.110286.
- [45] Wang, X., Han, Y., Leung, V.C.M., Niyato, D., Yan, X., Chen, X., 2020. Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 22, 869–904. doi:10.1109/COMST.2020.2970550.
- [46] World Wide Web Consortium, 2022. Decentralized identifiers (DIDs) v1.0. W3C Recommendation, 19 July 2022. Available at <https://www.w3.org/TR/did-core/>.
- [47] World Wide Web Consortium, 2024. Bitstring status list v1.0. W3C Candidate Recommendation Snapshot, 5 March 2024. Available at <https://www.w3.org/TR/vc-bitstring-status-list/>.
- [48] Xu, R., Chen, Y., Blasch, E., Chen, G., 2018. BlendCAC: A blockchain-enabled decentralized capability-based access control for IoTs, in: *Proceedings of the 2018 IEEE International Conference on Internet of Things (iThings)*, pp. 1027–1034. doi:10.1109/Cybermatics_2018.2018.00191.
- [49] Ye, N., Zhu, Y., Wang, R.c., Malekian, R., Lin, Q.m., 2014. An efficient authentication and access control scheme for perception layer of internet of things. *Applied Mathematics & Information Sciences* 8, 1617–1624. doi:10.12785/amis/080416.
- [50] Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I., 2019. HotStuff: BFT consensus with linearity and responsiveness, in: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 347–356. doi:10.1145/3293611.3331591.
- [51] Zaidi, S.M.S., Al-Qurishi, M., Abidin, M.Z.u., Almakdi, S., Asiri, Y., 2023. Fabrication: A hyperledger fabric framework for scalable IoT access control. *IEEE Internet of Things Journal* 10, 12636–12649. doi:10.1109/JIOT.2023.3254978.
- [52] Zhang, N., Wang, M., Wang, N., 2002. Precision agriculture: A worldwide overview. *Computers and Electronics in Agriculture* 36, 113–132. doi:10.1016/S0168-1699(02)00056-X.